

Form Container Block

1. Overview

This document defines the **Form Container block** for AEM **Edge Delivery Services (EDS)** with **Universal Editor (xWalk)**.

- It replaces the **AEM 6.4/6.5 Form Container component and its child form components** (Text, Dropdown, Hidden, File Upload, Captcha, etc.) with a **single EDS block** that owns:
 - Form layout and structure
 - Authoring of child form fields
 - Client-side validation, prefill, rules, multi-step logic
 - Integration with existing **middleware-based submission and file upload** flows

The goal is to preserve **existing business behavior** and authoring intent, while aligning with the **EDS block model** and client-side rendering.

2. Architecture: AEM vs EDS

2.1 Current State :

- **Form Container component** is a **parsys-style container**.
- Each field type is a **separate AEM component**, e.g.:
 - Text Input, Email, Phone
 - Dropdown
 - Hidden Field
 - File Upload
 - Captcha
 - Submit/Reset Button
- Each component has:
 - Its own **cq:dialog** (Coral UI)
 - Its own **Sling Model + HTL** rendering
 - Optional **dynamic dialog behavior** (datasource servlets, conditional show/hide, etc.)
- Rendering is **server-side** (HTL + Sling Models).

2.2 Target State – EDS with Universal Editor (xWalk)

- A single **form-container block** under **/blocks/form-container/** acts as the **parent form**.

- Inside `form-container` , authors configure **child items** (logical field types), e.g.:
 - `form-config` – form-level configuration
 - `form-input` – text/email/phone/number/date inputs
 - `form-options` – dropdowns, radio groups, checkbox groups
 - `form-textarea` – multi-line text
 - `form-hidden` – hidden values
 - `form-upload` – file upload
 - `form-button` – submit/reset
 - `form-label` – static instructional text
 - `form-fragment` – reusable field groups
 - `form-recaptcha` – reCAPTCHA
- All child types are defined as **component models** (for UE properties)
- Rendering is **client-side** via `form-container.js` :
 - Reads the authored model
 - Builds `<form>` HTML
 - Attaches validation, prefill, rules, multi-step, middleware integration

No server-side HTL/Sling models are used for form fields in EDS.

2.3 Structural Difference Summary

Aspect	AEM 6.4/6.5	EDS (xWalk)
Container	Form Container (parsys)	<code>form-container</code> block
Children	Separate AEM components	Child item types inside <code>form-container</code>
Rendering	Server-side (HTL + Sling Models)	Client-side (<code>form-container.js</code>)
Authoring UI	Coral dialogs (dynamic)	Universal Editor models (static JSON)
Code locations	Multiple OSGi bundles	Single <code>/blocks/form-container/</code> folder

Config	cq:dialog per component	component-models.json , component-definition.json , component-filters.json
--------	-------------------------	--

3. Authoring Experience in Universal Editor

3.1 How Authors Build a Form

1. Author adds a **Section** to the page.
2. Inside the Section, author drops a **Form Container** block from the component palette.
3. Inside the Form Container, author adds **child items** from a curated set:
 - **Form Config** – form-level settings (form ID, action type/middleware, redirect/thank-you, wizard step titles).
 - **Input Field** – single-line inputs (text, email, phone, number, date).
 - **Options Field** – dropdown, radio buttons, or checkbox groups.
 - **Text Area** – multi-line text.
 - **Hidden Field** – technical/context values (sessionId, formId, product IDs, ERP info, URL/cookie/timestamp).
 - **Upload Field** – file attachment selector (multi-file supported via middleware).
 - **Form Button** – submit/reset (or wizard navigation handled centrally).
 - **Form Label** – static instruction/description/disclaimer text.
 - **Form Fragment** – reusable group of fields sourced from another form page.
4. Each child item has its own **properties panel** in UE for label, name, type, required, step, layout span, etc.
5. Cross-cutting behavior (validation, prefill, rules, multi-step) is configured at form-level or via dedicated configuration (e.g., rules sheet), not per component code.

3.2 Allowed Child Items (Filter)

`component-filters.json` (or equivalent) will constrain which child types can appear inside `form-container` , for example:

```

1 {
2   "id": "form-container",
3   "components": [
4     "form-config",
5     "form-input",

```

```

6      "form-options",
7      "form-textarea",
8      "form-hidden",
9      "form-upload",
10     "form-button",
11     "form-label",
12     "form-fragment"
13   ]
14 }
15

```

- **Only** these child items are allowed inside `form-container`.
- No generic content blocks or unrelated components can't be nested inside.

3.3 Static Model Limitation & Extensions

- **Universal Editor models are static** (defined via JSON), unlike Coral dialogs which could:
 - Call servlets for dynamic dropdowns
 - Show/hide dialog fields based on other fields at dialog time
- For simple needs:
 - Static text, checkboxes, selects with fixed options → handled natively in the model
- For dynamic dialog behavior (e.g., GCMS form ID lists, Eloqua form IDs, advanced conditional UI in the authoring panel), an **App Builder UI Extension** will be required.

i Authoring remains **no-code** for authors, but dynamic configuration UIs (e.g., “pick GCMS form ID from API”) will be provided via a UI Extension, not via Coral dialogs.

4. Form Container Block Design

4.1 Form Container Block – Responsibilities

Block: `form-container`

Location: `/blocks/form-container/`

The Form Container:

- Declares and renders a single `<form>` element for the block.
- Reads and interprets all **child items** to:
 - Render form fields, sections, and labels
 - Apply layout (12-column grid)
 - Group fields into **steps** (multi-step wizard)
- Implements **client-side behavior**:
 - Validation (required, type, pattern, etc.)
 - Conditional rules (show/hide, enable/disable, etc.)

- Prefill (user profile, product/catalog context)
- reCAPTCHA integration (if enabled)
- File upload orchestration (browser → middleware)
- Submission orchestration:
 - Session ID retrieval
 - `getSecurityToken`
 - `saveFormData` payload construction and submission
- Does **not** persist data or files; all persistence remains in middleware.

4.2 Logical Child Item Types (Behavioral Summary)

Each child item is a **logical type**; exact UE model fields can be referenced from their AEM equivalent component.

Child Item	Purpose / Behavior	AEM Equivalent
Form Config	Form-level metadata and behavior: form ID, integration type/middleware target, success redirect/thank-you, wizard step titles.	Form Container dialog (action, form ID, success handling)
Input Field	Single-line input for text-like data: names, email, phone, numbers, dates. Supports required/format validation, prefill, optional step assignment and layout span.	Text Input, Email, Phone, Number, Date components Form Input
Options Field	Choice fields: dropdown (<code><select></code>), radio group, checkbox group. Options can be	Dropdown, Radio Group, Checkbox Group Form Options

	static or dynamic (Content Fragments, migrated ACS Generic Lists). Supports required, prefill, dependency rules (e.g., country → state).	
Hidden Field	Invisible fields used to carry context/technical values (sessionId, formId, actionType, catalog IDs, ERP info, URL/cookie/timestamp values). Populated at runtime and sent to middleware.	Hidden Field component + programmatic hidden inputs Form Hidden
Upload Field	File selector (single or multi-file). On file selection, the browser uploads directly to middleware and associates files with <code>sessionId</code> . No file storage in EDS.	File Upload component Form File Upload
Form Button	Submit/reset triggers. In single-step forms, authored submit buttons are used. In multi-step forms, navigation (Back/Next/Submit) is rendered by the container itself.	Submit/Reset buttons Form Button
Form Label	Static text or rich text inside the form	Static/Rich Text in forms

	(instructions, disclaimers, grouping titles). Does not submit a value.	Form Label
Form Fragment	Reference to a reusable group of fields defined on another form page (EDS fragment pattern). Allows reuse of common field groups, e.g. contact address block.	Experience Fragment inclusion / reusable field groups Form Experience Fragment Inclusion
Form reCAPTCHA	recaptcha capabilities	Form reCAPTCHA

i Each child item focuses on **capturing/displaying its own value and metadata**. Cross-cutting behavior (validation, rules, prefill, multi-step, submission) is centrally managed by the **Form Container**.

5. Multi-Step Wizard Behavior

- Any field or label can be assigned a **Wizard Step** (Step 1, 2, 3, ...) via configuration.
- The Form Container:
 - Groups fields into steps.
 - Renders a **progress indicator** (steps with titles).
 - Shows only the current step; others are hidden.
 - Renders navigation controls:
 - Next/Back** for intermediate steps.
 - Submit** on the final step.
- Validation:**
 - On **Next**, only fields in the current step are validated.
 - On **Submit**, all steps are validated.
 - Back** does not trigger validation.
- Step titles are authored at form-level (e.g., “Contact Info”, “Preferences”, “Review”).